Learning fixed-complexity polyhedral Lyapunov functions from counterexamples

Guillaume O. Berger and Sriram Sankaranarayanan

***Abstract*— In this paper, we present an algorithm for synthesizing polyhedral Lyapunov functions for hybrid systems with multiple modes, each with linear dynamics and state-based switching between these modes. The problem of proving global asymptotic stability (GAS) for such systems is quite challenging. In this paper, we present an algorithm to synthesize a fixed-complexity polyhedral Lyapunov function to prove that such a system is GAS. Such functions are defined as the piecewise maximum over a fixed number of linear functions. Previous work on this problem reduces to a system of *bilinear* constraints that are well-known to be computationally hard to solve precisely. Therefore, heuristic approaches such as alternating descent have been employed. In this paper, we first prove that deciding if there exists a polyhedral Lyapunov function for a given piecewise linear system is a NP-hard problem. We then present a counterexample-guided algorithm for solving this problem. Our approach alternates between choosing candidate Lyapunov functions based on a finite set of counterexamples, and verifying if these candidates satisfy the Lyapunov conditions. If the verification of a given candidate fails, we find a new counterexample that is added back to our set. We prove that if this algorithm terminates, it discovers a valid Lyapunov function or concludes that no such Lyapunov function exists. However, our initial algorithm can be non-terminating. We modify our algorithm to provide a terminating version based on the so-called cutting-plane argument from nonsmooth optimization. We demonstrate our algorithm on small numerical examples.**

I. Introduction

Lyapunov methods are useful tools for dynamical systems analysis [1], [2]. Among the various classes of functions studied for computational Lyapunov analysis, the one of polyhedral functions (convex piecewise linear functions) is particularly appealing due to its expressiveness. Indeed, the expressiveness of the function can be easily modulated by adjusting the number of linear pieces, and they are asymptotically nonconservative for the stability analysis of a large class of classical and hybrid dynamical systems, including switched linear systems [3]. The computation of polyhedral Lyapunov functions nevertheless remains difficult, even for linear time-invariant systems. The main difficult arises from the fact that the requirement for the function to decrease along the trajectories of the system gives rise to nonlinear, nonconvex constraints [4].

In this paper, we establish that the problem of checking if there exists a fixed complexity polyhedral Lyapunov function for a continuous-time, piecewise linear dynamical system is NP-hard. We introduce an active learning framework to compute polyhedral Lyapunov functions for such systems. The approach alternates between a learning step and a verification step. In the learning step, a candidate polyhedral function is computed based a finite set of counterexamples. Then, in the verification step, one checks whether the candidate function is a valid Lyapunov function for the system, and if not, outputs a new counterexample point. Our approach proceeds to use the new counterexample to *remove* the previous candidate (and many others) from further consideration. The process then proceeds until either a valid Lyapunov function is found or no such function is shown to exist for the given system.

The key challenges in this approach are two-fold: (a) At each step, there are multiple possible refutations for a given counterexample. Our approach uses a tree-based search algorithm to explore each possible refutation; (b) The algorithm as presented would be non-terminating. We modify our approach to guarantee termination using the *cutting-plane argument*: wherein a careful choice of candidate Lyapunov function at each step and pruning away branches of the tree based on the volume of possible candidates are used to ensure termination and prove upper bounds on the running-time complexity of the algorithm.

Comparison with the literature. The idea of learning Lyapunov functions from data and verifying the result has been explored by many researchers, starting with Topcu et al. [5]. However, their approach relies on random sampling of states with sum-of-squares (SOS) programming to verify the result. In contrast, our approach iteratively adds samples obtained as counterexamples from a verification problem. Deshmukh et al. present an approach for finding Lyapunov function for nonlinear systems that combines learning from finitely many samples and using a global optimization to generate more samples based on a current candidate in addition to an exhaustive verification procedure [6]. Chang et al. present a counterexample-driven approach for inferring neural networks as Lyapunov functions simultaneously with neural network controllers for dynamical systems [7]. Their approach uses back-propagation for learning neural network and a interval-based nonlinear solver dReal for verifying the result. However, there are no guarantees that these approaches would find a Lyapunov function, even if one exists. Ravanbakhsh et al. present a technique for deriving polynomial Lyapunov functions using sum-of-squares (SOS) optimization for nonlinear systems that is based on generating candidate functions from a set of unexplored candidates and verifying them using SOS optimization [8]. At a high level, this paper employs a similar strategy to generate polyhedral Lyapunov functions. However, unlike Ravanbakhsh et

G. Berger is a BAEF fellow. Funding information will be added to final version.Both authors are affiliated with the University of Colorado Boulder. Emails: {guillaume.berger,srirams}@colorado.edu

al., our work focuses on linear hybrid systems and does not resort to any relaxations in our verification step. Due to the linearity of the dynamics in each mode, verification requires solving linear programs rather than SOS programs. On the other hand, our approach requires us to consider multiple ways of refuting a given counterexample which is tracked using a tree of possible scenarios.

Methods for the computation of polyhedral Lyapunov functions for linear dynamics can be divided into set-theoretic and optimization-based methods. Set-theoretic methods [4], [9], [10] typically proceed by computing recursive images of a polyhedron in the state space until it eventually provides a polyhedral Lyapunov function for the system. An important drawback of these methods is that they are restricted to discrete-time systems and often lack complexity bounds. Optimization-based methods [11]–[13] aim to solve the nonlinear, nonconvex optimization problem accounting for the existence of a polyhedral Lyapunov function. This is achieved, for instance, by considering convex relaxations of this problem [12], arbitrarily fixing some variables [11], or using an alternating descent procedure to solve locally the optimization problem [13]. These methods are not guaranteed to converge to a global optimum of the optimization problem, so that they may fail to find a polyhedral Lyapunov function for the system, even if one exists. Our approach borrows from both of the above ones: we use an iterative process to recursively update the candidate Lyapunov function, and the update is done by solving optimization problems. The approach comes with formal guarantees of complexity and convergence to a global optimum.

Organization. Section II introduces the problem of interest and preliminary concepts related to Lyapunov analysis and polyhedral functions. In Section III, we describe the algorithm and its properties. Finally, in Section IV, we demonstrate the applicability of the process on a numerical example.

Notation. $\mathbb{N}$ denotes the set of nonnegative integers. We let $\mathbb{N}_* = \mathbb{N} \setminus \{0\}$ and $\mathbb{R}^d_* = \mathbb{R}^d \setminus \{0\}$. For $n \in \mathbb{N}$, we let $[n] = \{1, \dots, n\}$. For $x \in \mathbb{R}^d$, $\|x\|$ is the ∞-norm of x (max of the absolute value of the entries).

II. Preliminaries and Problem Description

Let Q be a nonempty finite set, and for each $q \in Q$, let $\mathcal{H}_q \subseteq \mathbb{R}^d$ be a closed polyhedral cone described by $\mathcal{H}_q = \{x \in \mathbb{R}^d : M_q x \geq 0\}$ where $M_q \in \mathbb{R}^{n_q \times d}$, $n_q \in \mathbb{N}$. We let $\mathcal{Q} : \mathbb{R}^d \rightrightarrows Q$ be the set-valued function defined by $\mathcal{Q}(x) = \{q \in Q : x \in \mathcal{H}_q\}$. Finally, for each $q \in Q$, let $A_q \in \mathbb{R}^{d \times d}$.

Definition 1 (Piecewise linear dynamical system): Given a set-valued map $\mathcal{Q}$ and matrices $(A_q)_{q \in Q}$, the corresponding piecewise linear dynamical system is described by the dynamics:

$$\xi'(t) \in \{A_q \xi(t) : q \in \mathcal{Q}(\xi(t))\}. \tag{1}$$

A function $\xi : \mathbb{R}_{\geq 0} \to \mathbb{R}^d$ is called a *trajectory* of System (1) if ξ is absolutely continuous and satisfies the differential inclusion (1) for almost all $t \in \mathbb{R}_{\geq 0}$ [14].

We aim to study the stability of the origin for System (1) using Lyapunov functions.

Definition 2 (Lyapunov function):

- A continuous function $V : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ is a *candidate Lyapunov function* if (i) $V(0) = 0$ and for all $x \in \mathbb{R}^d_*$, $V(x) > 0$, and (ii) V is radially unbounded[1].
- A candidate Lyapunov function $V : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ is a *Lyapunov function* for System (1) if for all $x \in \mathbb{R}^d_*$, $q \in \mathcal{Q}(x)$ and $v = A_q x$,

$$D^+ V(x; v) \triangleq \limsup_{s \to 0^+} \frac{V(x + sv) - V(x)}{s} < 0.$$

If System (1) admits a Lyapunov function, then $x = 0$ is an asymptotically stable equilibrium [1, Theorem 4.2]. We consider *polyhedral* Lyapunov functions, which are defined as the pointwise maximum of a finite set of linear functions.

Definition 3 (Polyhedral function): Let $(c_i)_{i=1}^m \subseteq \mathbb{R}^d$. The *polyhedral function* associated to $(c_i)_{i=1}^m$ is the function $V : \mathbb{R}^d \to \mathbb{R}$ defined by $V(x) = \max_{i \in [m]} c_i^\top x$.

Proposition 1: Let $(c_i)_{i=1}^m \subseteq \mathbb{R}^d$. The polyhedral function V associated to $(c_i)_{i=1}^m$ is a Lyapunov function for System (1) iff

(C1) for all $x \in \mathbb{R}^d_*$, $V(x) > 0$;

(C2) for all $x \in \mathbb{R}^d_*$, $q \in \mathcal{Q}(x)$ and $i \in [m]$ with $c_i^\top x = V(x)$, it holds that $c_i^\top A_q x < 0$.

Proof: The fact that (C1) is equivalent to V being a candidate Lyapunov function is straightforward. Now, if V is a candidate Lyapunov function, the fact that (C2) is equivalent to V being a Lyapunov function for System (1) follows from the fact that for any $x, v \in \mathbb{R}^d$, $D^+ V(x; v) = \max\{c_i^\top v : c_i^\top x = V(x),\, i \in [m]\}$ [4, Eq. 2.30]. ■

It follows that a set a vectors $(c_i)_{i=1}^m \subseteq \mathbb{R}^d$ represents a polyhedral Lyapunov function for System (1) iff

$$\begin{aligned} &(\forall\, x \in \mathbb{R}^d_*,\, i \in [m],\, q \in Q) \\ &\quad [\,(x \in \mathcal{H}_q \;\wedge\; (\forall\, j \in [m])\; c_i^\top x \geq c_j^\top x) \\ &\qquad\qquad \Rightarrow\; (c_i^\top x > 0 \;\wedge\; c_i^\top A_q x < 0)\,]. \end{aligned} \tag{2}$$

Given a piecewise linear system (1) and a desired number of pieces $m \geq 2$, the m-Lyap-Fun-Exists problem asks whether the conditions in (2) hold for some $(c_i)_{i=1}^m \subseteq \mathbb{R}^d$.

Theorem 2: The m-Lyap-Fun-Exists problem is NP-hard, even for $m = 2$.

Proof: See Appendix A. ■

A. Optimization Problem Formulation

Condition (2) involves an infinite set of constraints in $(c_i)_{i=1}^m$ because it must hold for all $x \in \mathbb{R}^d_*$. Using duality of Linear Programming [15], one can transform (2) into a finite set of constraints. Indeed, the feasibility of (2) is equivalent to the existence of Lagrange multipliers $(\Lambda_{iq})_{i \in [m], q \in Q} \in \prod_{i \in [m], q \in Q} \mathbb{R}^{n_q}_{\geq 0}$, $(\mu_{ijq})_{i,j \in [m], q \in Q} \subseteq \mathbb{R}_{\geq 0}$, $(\nu_i)_{i=1}^m \subseteq \mathbb{R}^{2d}_{\geq 0}$ and $\eta > 0$ such that

$$\begin{aligned} &\textstyle\sum_{i=1}^m c_i \nu_i^\top = [I_d, -I_d], \\ &(\forall\, i \in [m],\, q \in Q) \\ &\quad c_i^\top A_q + \eta c_i^\top = \textstyle\sum_{i=j}^m \mu_{ijq}(c_j^\top - c_i^\top) - \Lambda_{iq} M_q. \end{aligned} \tag{3}$$

[1] A function $V : \mathbb{R}^d \to \mathbb{R}$ is said to be *radially unbounded* if $\lim_{r \to \infty} \inf\{V(x) : x \in X,\, \|x\| \geq r\} = \infty$.

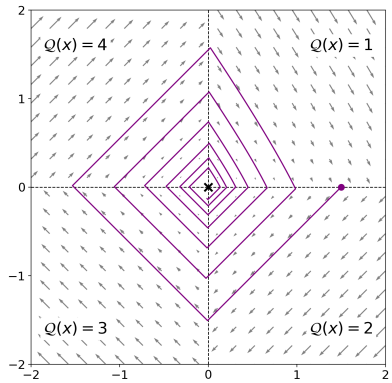

Fig. 1. Piecewise linear system described in Example 1. The system consists in 4 pieces corresponding to the 4 orthants of $\mathbb{R}^2$. The vector field is represented by gray arrows, and a sample trajectory starting from $x = [1.5, 0]^\top$ is represented in purple.

Condition (3) contains a finite set of constraints. However, the constraints are bilinear in the decision variables $(c_i)_{i=1}^m$, $(\Lambda_{iq})_{i\in[m],q\in Q}$, $(\mu_{ijq})_{i,j\in[m],q\in Q}$, $(\nu_i)_{i=1}^m$ and η. Bilinear optimization programs are in general NP-hard to solve [16]. An alternating descent procedure to solve a program similar to (3) is proposed in [13]. An important drawback of this approach is that the convergence to a feasible point is not guaranteed. In fact, alternating descent often has the drawback of getting "stuck" at saddle points from whence no further solution improvement is possible [17].

In this paper, we focus on condition (2). We tackle the fact that (2) involves an infinite set of constraints by sampling a finite subset of these constraints and solving the associated sampled feasibility problem. We refine iteratively the solution of the sampled problem by adding new constraints to it, in a counterexample-guided fashion. The process is guaranteed to converge to a feasible solution of (2), if one exists.

We describe the refinement process and discuss its complexity in the next section. We will use the following running example to illustrate the approach.

Example 1 (Running illustrative example): Consider the continuous-time piecewise linear system described by (1) with $Q = \{1, \dots, 4\}$, $H_1 = \mathbb{R}_{\geq 0} \times \mathbb{R}_{\geq 0}$, $H_2 = \mathbb{R}_{\geq 0} \times \mathbb{R}_{\leq 0}$, $H_3 = \mathbb{R}_{\leq 0} \times \mathbb{R}_{\leq 0}$ and $H_4 = \mathbb{R}_{\leq 0} \times \mathbb{R}_{\geq 0}$, and

$$A_1 = \begin{bmatrix} \frac{1}{2} & 1 \\ -1 & \frac{-3}{2} \end{bmatrix}, \quad A_2 = A_4 = \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix},$$
$$A_3 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}.$$

The vector field and a trajectory of this system starting from $x = [1.5, 0]^\top$ are represented in Figure 1. This system does not admit a polynomial Lyapunov function (a proof is provided in Appendix B). Nevertheless, as we will see throughout the paper, we can compute a polyhedral Lyapunov function for the system, thereby ensuring that it is asymptotically stable.

III. Counterexample-guided Search

The process involves a *learning phase* and a *verification* phase, that are performed in alternation. Our approach maintains a *tree* of *scenarios* wherein each scenario is defined by a finite set of triples of the form $Y = \{(x_1, i_1, j_1), \dots, (x_N, i_N, j_N)\}$ wherein for each $l \in [N]$, $x_l \in \mathbb{R}^d_*$ is a state with associated "book-keeping" indices $i_l, j_l \in [m]$ whose meaning will be explained subsequently.

At each step, our approach picks a leaf of this tree and expands it as follows:

1) *Learning:* Solve a linear program to obtain a candidate Lyapunov function represented by $(c_i)_{i=1}^m$.
2) *Verification:* The candidate solution is then fed to a verifier that checks whether it provides a valid solution to (2) for all states $x \in \mathbb{R}^d_*$ and $i \in [m]$. If this fails, the verifier provides new counterexample (x, i), indicating that the state $x \in \mathbb{R}^d_*$ fails to satisfy (2) wherein $i \in [m]$ is a piece of the polyhedral Lyapunov function that is maximal at x.
3) *Refutation:* In response to the counterexample, our approach now considers possible ways to *refute* this counterexample. In particular, there are m possible refutations. Each refutation either contradicts the fact that the ith piece is maximal by asserting that $c_j^\top x > c_i^\top x$, or if $i = j$, the refutation asserts the Lyapunov decrease conditions are now enforced to hold at x. In all cases, the purpose of the refutation is to eliminate the previous candidate solutions and a set of related solutions from further consideration.

This process yields a tree of possible scenarios. Our approach systematically explores this tree and stops when one of the leaves of the tree yields a valid Lyapunov function. Under some modifications of the basic scheme above, we prove that the depth of this tree will be bounded. The algorithm is guaranteed to find a polyhedral Lyapunov function if one exists, or conclude that no such function exists. This yields an upper bound on the running time complexity of the algorithm.

The section is organized as follows: first, we describe the learning phase, then the verification phase, and finally the refutation which yields an overall recursive process.

A. Learning Phase

Let us define a predicate: $\mathbb{P}(x, i, j)$ for $x \in \mathbb{R}^d$, $i, j \in [m]$:

$$\mathbb{P}(x,i,j) = \begin{cases} c_j^\top x > c_i^\top x & \text{if } j \neq i, \\ c_i^\top x > 0 \wedge & \\ \quad (\forall\, q \in \mathcal{Q}(x))\, c_i^\top A_q x < 0 & \text{if } j = i. \end{cases} \tag{4}$$

In the above, the coefficients $(c_i)_{i=1}^m$ are understood from the context; when confusion is possible, we add them to the arguments: $\mathbb{P}(x, i, j; (c_i)_{i=1}^m)$. Note that condition (2) is equivalently written as:

$$(\forall x \in \mathbb{R}^d_*,\ i \in [m])\ (\exists j \in [m])\ \mathbb{P}(x, i, j).$$

In the learning phase, given a *scenario* $Y \subseteq \mathbb{R}^d_* \times [m] \times [m]$, we aim to find vectors $(c_i)_{i=1}^m \subseteq \mathbb{R}^d$ satisfying $\mathbb{P}(x, i, j)$ for all $(x, i, j) \in Y$. Therefore, we consider the following feasibility problem:

$$\text{find} \quad (c_i)_{i=1}^m \subseteq \mathbb{R}^d \tag{5a}$$

$$\text{s.t.} \quad \mathbb{P}(x,i,j), \quad \forall\,(x,i,j) \in Y, \tag{5b}$$
$$\|c_i\| \le \tfrac{1}{2}, \quad \forall\, i \in [m]. \tag{5c}$$

Any feasible solution to Problem (5) will be called a *candidate solution according to scenario* Y. When Y is finite, Problem (5) can be solved efficiently using Linear Programming (with strict inequalities) [18], [19].

B. Verification Phase

In the verification phase, we use the candidate solution $(c_i)_{i=1}^m$ computed in the learning phase and verifies whether the polyhedral function V associated to $(c_i)_{i=1}^m$ provides a valid Lyapunov function for System (1), by checking whether conditions (C1) and (C2) in Proposition 1 are satisfied. If this is not the case, the verifier generates a pair $(x,i) \in \mathbb{R}^d \times [m]$, called a *counterexample*, such that for all $j \in [m]$, $\mathbb{P}(x,i,j)$ does not hold.

First, we verify that (C1) holds for the candidate solution by searching for $x \in \mathbb{R}^d_*$ such that $V(x) \le 0$. Without loss of generality, we could set some component $x_k \in \{-1,1\}$ while allowing the remaining components of x to be chosen arbitrarily. We solve $2d$ linear programming problem (LP), for each $k \in [d]$ and $s \in \{-1,1\}$,

$$\text{find} \quad x \in \mathbb{R}^d \tag{6a}$$
$$\text{s.t.} \quad c_i^\top x \le 0, \quad \forall\, i \in [m], \tag{6b}$$
$$x_k = s. \tag{6c}$$

If for all k, s, (6) is infeasible, then (C1) holds. However, if there are k, s for which (6) has a feasible solution x, then it holds that $x \neq 0$ and $V(x) \le 0$. We find index $i \in [m]$ with $c_i^\top x = V(x)$, yielding a counterexample (x,i).

If all the problems in (6) are infeasible, we next verify that (C2) holds for the candidate solution. Therefore, we solve $m|Q|$ LPs, for each $i \in [m]$ and $q \in Q$,

$$\text{find} \quad x \in \mathbb{R}^d \tag{7a}$$
$$\text{s.t.} \quad c_i^\top x = 1 \ge c_j^\top x, \quad \forall\, j \in [m], \tag{7b}$$
$$x \in \mathcal{H}_q, \tag{7c}$$
$$c_i^\top A_q x \ge 0. \tag{7d}$$

If for all i, q, (7) is infeasible, then (C2) holds. However, if there are i, q for which (7) has a feasible solution x, then it holds that $x \neq 0$, $q \in \mathcal{Q}(x)$, $c_i^\top x = V(x)$ and $c_i^\top A_q x \ge 0$, so that (x,i) is a counterexample.

Example 2 (Running illustrative example): Consider the 4-piece polyhedral function V whose 1-sublevel set is represented in Figure 2-left (in yellow). The point $x = [-1, 0]^\top$ (in red) provides a direction in which αx is in the 1-sublevel set of V for all $\alpha \ge 0$. This implies that $V(x) \le 0$, so that x is a feasible solution to (6).

Now, consider the system of Example 1 and consider the 10-piece polyhedral function V whose 1-sublevel set is represented in Figure 2-right (in yellow). Problem (7) with $q = 4$ has a feasible solution $x = [0, 1.2]^\top$ (red dot). The flow direction $A_4 x$ of the system from x is represented in Figure 2-right (red line). We see that the flow direction points towards the exterior of the 1-sublevel set of V.

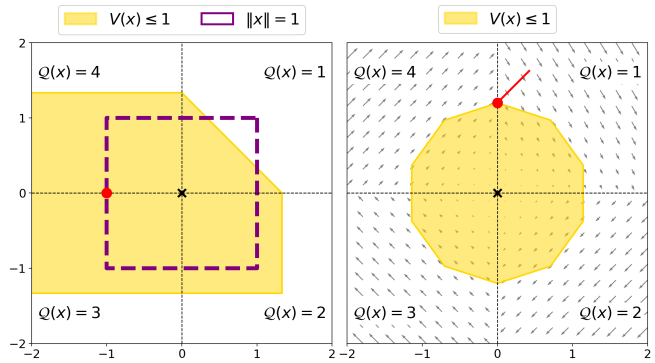

Fig. 2. *Left:* A point $x \in \mathbb{R}^d_*$ (in red) at which $V(x) \le 0$. *Right:* A point $x \in \mathbb{R}^d_*$ (in red) at which $D^+V(x; A_q x) \ge 0$ for the system described in Example 1.

C. Refutation and Overall Algorithm

Recall that a *scenario* is a finite set of triples of the form $Y = \{(x_1, i_1, j_1), \ldots, (x_N, i_N, j_N)\}$ and with each scenario, we associate a convex set of candidates $S(Y)$ according to (5). The algorithm organizes the scenarios as a tree with the following properties: (a) each node v is associated with a scenario $Y(v)$; (b) the root of the tree is associated with $Y = \emptyset$ and (c) each node v has m children, $v_1, \ldots, v_m$, wherein $Y(v_j) = Y(v) \uplus \{(x_{N+1}, i_{N+1}, j)\}$. Each leaf of the tree is marked as UNEXPLORED, or INFEASIBLE.

Each iteration of our algorithm is as follows:

1) Choose an UNEXPLORED leaf u with associated scenario Y.
2) Find a candidate $(c_i)_{i=1}^m \in S(Y)$.
3) Verify the candidate using (6) and (7).
4) If we found a counterexample (x,i), we add m children to u wherein u_j is UNEXPLORED and has the scenario $Y_j = Y \uplus \{(x,i,j)\}$.

Figure 3 illustrates this process and Algorithm 1 shows the pseudo-code. We use the notation $S(v) = S(Y(v))$. We will now establish some of the main properties of our algorithm. Let $(c_i)_{i=1}^m$ be a candidate chosen for some leaf node u (Line 7) and (x,i) be a counterexample that was found corresponding to the candidate (Line 11). Consider $S(u_j)$ for child u_j for some $j \in [m]$ (Line 13).

Proposition 3: $S(u_j) \subseteq S(u)$ and $(c_i)_{i=1}^m \notin S(u_j)$.

Proof: Recall from (5): $S(Y) = \{(c_i)_{i=1}^m : [(\forall\, i \in [m])\ \|c_i\| \le \frac{1}{2}] \wedge [(\forall\,(x,i,j) \in Y)\ \mathbb{P}(x,i,j)]\}$. Since $Y(u_j) \supseteq Y(u)$, it follows that $S(u_j) \subseteq S(u)$.

To prove strict inclusion, we now prove that $(c_i)_{i=1}^m \notin S(u_j)$. Note that $Y(u_j) = Y(u) \uplus (x,i,j)$ where (x,i) is a counterexample to $(c_i)_{i=1}^m$. In other words, either (Case-A) some LP in (6) is feasible, implying that there is $i \in [m]$ and $x \neq 0$ such that

$$[(\forall j \in [m])\ c_i^\top x \ge c_j^\top x] \wedge c_i^\top x \le 0,$$

or (Case-B) some LP in (7) is feasible for some $i \in [m]$ and $q \in Q$:

$$[(\forall j \in [m])\ c_i^\top x = 1 \ge c_j^\top x] \wedge x \in \mathcal{H}_q \wedge c_i^\top A_q x \ge 0.$$

Algorithm 1: Find Polyhedral Lyapunov Function.

Data: System (1), number of pieces m.
Result: $(c_i)_{i=1}^m$: polyhedral Lyapunov function coefficients.

1 Initial tree is set to a root r with $Y(r) = \emptyset$.
2 **while** *Tree has* UNEXPLORED *leaves* **do**
3 Choose UNEXPLORED leaf u.
4 Formulate the constraints $S(u)$ ((5)).
5 **if** $S(u)$ *infeasible* **then** mark u as INFEASIBLE.
6 **else**
7 Choose $(c_i)_{i=1}^m \in S(u)$
8 Verify $(c_i)_{i=1}^m$ ((6) and (7)).
9 **if** $(c_i)_{i=1}^m$ *is Lyapunov* **then** return $(c_i)_{i=1}^m$
10 **else**
11 Let (x, i) be the counterexample.
12 Add m children $u_1, \ldots, u_m$ to u.
13 **for** $j = 1, \ldots, m$ **do**
$Y(u_j) \leftarrow Y(u) \cup \{(x, i, j)\}$.
14 Mark each u_j as UNEXPLORED.
15 Return "No Lyapunov Found"

Observe that $\mathbb{P}(x, i, j)$ is not satisfied by $(c_i)_{i=1}^m$. If $i \neq j$, $\mathbb{P}(x, i, j) = c_j^\top x > c_i^\top x$, which is not true for both Case-A and Case-B. On the other hand, if $i = j$, $\mathbb{P}(x, i, j)$ requires $c_i^\top x > 0$ and $c_i^\top A_q x < 0$ for all $q \in Q$. In Case-A, the former condition is violated and in Case-B, the latter condition is violated. We thus conclude that $S(u_j)$ which requires $\mathbb{P}(x, i, j)$ cannot contain the candidate $(c_i)_{i=1}^m$. ■

Proposition 4: If $(\tilde{c}_i)_{i=1}^m \in S(u)$ but $(\tilde{c}_i)_{i=1}^m \notin S(u_j)$ for all $j \in [m]$, then the function represented by $(\tilde{c}_i)_{i=1}^m$ cannot be a Lyapunov function.

Proof: Recall from (5): $S(u) = \{(c_i)_{i=1}^m : [(\forall i \in [m]) \, \|c_i\| \leq \frac{1}{2}] \wedge [(\forall (x, i, j) \in Y(u)) \, \mathbb{P}(x, i, j)]\}$. Let (x, i) be the counterexample found by the verifier. Since for each $j \in [m]$, $Y(u_j) = Y(u) \uplus \{(x, i, j)\}$, it follows that for all $j \in [m]$, $\neg\mathbb{P}(x, i, j; (\tilde{c}_i)_{i=1}^m)$. Expanding the latter, we note that for all $j \in [m]$, $\tilde{c}_j^\top x \leq \tilde{c}_i^\top x$ but either $\tilde{c}_i^\top x \leq 0$ or $\tilde{c}_i^\top A_q x \geq 0$ for some $q \in Q$. In other words, the function represented by $(\tilde{c}_i)_{i=1}^m$ cannot be a Lyapunov function. ■

Let us assume that the underlying System (1) has a polyhedral Lyapunov function given by $(c_i^*)_{i=1}^m$ and without loss of generality assume $\|c_i^*\| \leq \frac{1}{2}$ for all $i \in [m]$.

Proposition 5: Let $L = \{v_1, \ldots, v_K\}$ be the set of all UNEXPLORED leaves of the tree during any iteration of Alg. 1. There exists $v_j \in L$ such that $(c_i^*)_{i=1}^m \in S(v_j)$.

Proof: The proof is by induction on the number of steps of the algorithm. The property is true at the very beginning since $(c_i^*) \in S(\text{root})$. Assume it is true after k iterations of the while loop. Let u be the unexplored root chosen to be expanded at this step. Suppose $u \neq v_j$. Then, we will trivially satisfy the property for the tree at the subsequent step. Otherwise, suppose $u = v_j$, but for the sake of contradiction, assume $(c_i^*)_{i=1}^m \notin S(u_j)$ for any of the children of u created at this step. However, by Proposition 4, any such solution

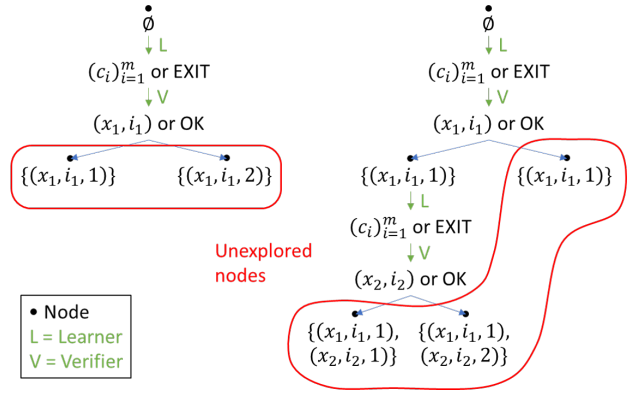

Fig. 3. The tree after steps 1 (left) and 2 (right) of the algorithm with $m = 2$. At step 2, the node $\{(x_1, i_1, 1)\}$ is picked and processed, leading to the tree on the right.

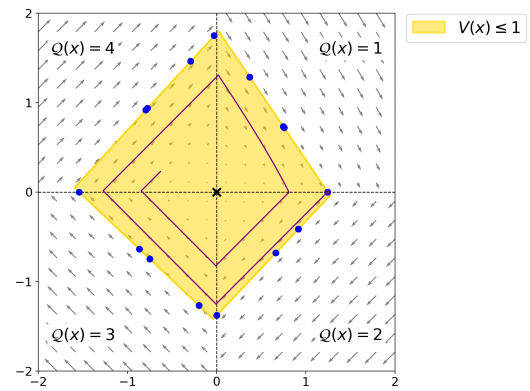

Fig. 4. 4-piece polyhedral Lyapunov function V for the system in Example 1. The blue dots are the points involved in the scenario leading to this polyhedral function. A sample trajectory starting from $x = [1.5, 0]^\top$ is represented in purple.

cannot be a Lyapunov function. This yields a contradiction. ■

Example 3 (Running illustrative example): Consider the system of Example 1. A 4-piece polyhedral Lyapunov function for this system was computed using the process described above. Its 1-sublevel set is represented in Figure 4 (in yellow). The points "x" involved in the scenario $Y \in \mathbb{R}_*^d \times [m] \times [m]$ leading to this function are represented by blue dots. We have also represented a trajectory of the system starting from $x = [1.25, 0]^\top$ (in purple). We observe that the trajectory does not leave the sublevel set, as expected.

We discuss the soundness and termination of Alg. 1.

Theorem 6: If Alg. 1 terminates and returns vectors $(c_i)_{i=1}^m$, then the polyhedral function associated to $(c_i)_{i=1}^m$ is a Lyapunov function for System (1). However, if Alg. 1 stops and returns "No Lyapunov Found", then System (1) does not admit an m-piece polyhedral Lyapunov function.

Proof: If Alg. 1 returns $(c_i)_{i=1}^m$, it means that $(c_i)_{i=1}^m$ has passed successfully the verification phase (Line 9), thereby ensuring that $(c_i)_{i=1}^m$ provides a Lyapunov function for System (1). Suppose Alg. 1 terminates without finding a Lyapunov function. Therefore, all leaves of the tree are marked infeasible. By applying Proposition 5, we note that

TABLE I

NOTIONS OF "CENTERS" FOR BOUNDED CONVEX SETS IN $\mathbb{R}^r$.

Choice of x	γ
Center of gravity [21]	0.63
Center of the Maximum Volume inscribed Ellipsoid [22]	$1-1/r$

no Lyapunov function could have existed in the first place. ■

Note, however, that there is no guarantee that the tree exploration process in Alg. 1 will terminate. To ensure termination, we modify the algorithm by using ideas from Ellipsoidal method. To show this, we leverage a central result in nonsmooth optimization, known as the *cutting-plane* argument. Therefore, consider Table I which provides two notions of "centers" of convex sets.

Lemma 7 (Cutting-plane argument [20]): Let $\mathcal{S} \subseteq \mathbb{R}^r$ be a bounded convex set. Let x be the "center" of $\mathcal{S}$ according to one of the definitions in Table I. Let $\mathcal{T}$ be a convex set not containing x. Then, it holds that $\mathrm{vol}(\mathcal{S} \cap \mathcal{T}) \leq \gamma \mathrm{vol}(\mathcal{S})$ where $\gamma \in (0,1)$ does not depend on $\mathcal{S}$ and $\mathcal{T}$.

Based on the above, consider the following modification of the algorithm. Fixed $\epsilon > 0$ and consider Alg. 1, except that in the learning phase, if $\mathrm{vol}(S(Y)) < \epsilon$, then u is declared INFEASIBLE; otherwise, we choose the candidate $(c_i)_{i=1}^m$ as the "center" of $S(Y)$ according to one of the definitions in Table 1 (with $r = md$). See Algorithm 2 for a pseudo-code of the modified learning phase. It holds that the modified algorithm terminates in finite time.

Algorithm 2: Modify Alg. 1 Line 7 for termination.

(7-1) **if** $\mathrm{vol}(S(u)) \leq \epsilon$ **then** mark u as INFEASIBLE;

(7-2) Choose $(c_i)_{i=1}^m$ as the "center point" of $S(u)$ (see Table I).

Theorem 8 (Termination and soundness of Alg. 2): Alg. 2 always terminates. Moreover, if Alg. 2 returns vectors $(c_i)_{i=1}^m$, then the polyhedral function associated to $(c_i)_{i=1}^m$ is a Lyapunov function for System (1).

Proof: First, we prove that Alg. 2 always terminates. Let u be a current leaf that is expanded and $u_1, \ldots, u_m$ be the children that are added. We will first prove that $\mathrm{vol}(S(u_j)) \leq \gamma \mathrm{vol}(S(u))$. First of all, by Proposition 3, we note that the candidate found at the current iteration satisfies $(c_i)_{i=1}^m \notin S(u_j)$. Since we choose (c_i) as the center of $S(u)$, by the cutting-plane argument (Lemma 7), $\mathrm{vol}(S(u_j)) \leq \gamma \mathrm{vol}(S(u))$. At the very start of the algorithm, we have $S(\text{root}) = \{(c_i)_{i=1}^m : \|c_i\| \leq \frac{1}{2}\}$ which has finite volume V_0. By induction, if v is an unexplored leaf at depth D, we can show that $\mathrm{vol}(S(v)) \leq \gamma^D V_0$. Since we terminate whenever $\mathrm{vol}(S(v)) \leq \epsilon$, the depth of the tree is bounded by $\frac{\log(\epsilon) - \log(V_0)}{\log(\gamma)}$. This proves that Alg. 2 eventually terminates.

Finally, the proof that Alg. 2 is sound is identical to the first part of the proof of Theorem 6. ■

Remark 1: By ensuring termination, we have to give up on the completeness of Alg. 2: i.e, there could exist a Lyapunov function $(c_i^*)_{i=1}^m$ but not found by Alg. 2. Stopping the algorithm when the volume of $S(u)$ becomes too small is relevant in practice, as numerical solvers cannot reliably decide feasibility when the volume of the feasible set is too small (typically of the order of the machine precision).

D. Complexity analysis

Note that, by definition of $S(\text{root})$, the initial volume V_0 satisfies $V_0 = 1$. The dominant factor in the complexity of Alg. 2 is the depth D of the tree explored during the running of the algorithm. From the proof of Theorem 8, it holds that

$$D \leq \frac{\log(\epsilon)}{\log(\gamma)} + 1, \tag{8}$$

where $\gamma \in (0,1)$ is the shrinking factor in Lemma 7. Hence, Alg. 2 explores at most m^{D+1} nodes before termination. Now, let us look at the complexity of exploring a single node. This requires to perform the learning phase and the verification phase for this node:

- The learning phase requires to find the "center" of a polyhedron in $\mathbb{R}^{md}$, described by $\mathcal{O}(D)$ constraints, according to one of the definitions in Table I. Computing the center of gravity is very challenging in general [20]. Therefore, we consider the center of the Maximum Volume inscribed Ellipsoid (MVE). Computing the MVE and its center can be achieved by using Semidefinite Programming [19, Proposition 4.9.1], so that it can be done in polynomial time w.r.t. md and D [19, §6.6.3]. Moreover, the volume of the MVE scaled by md is an upper bound on the volume of the polyhedron [23].
- The verification phase involves the resolution of $2d + m|Q|$ Linear Programs in $\mathbb{R}^d$ with at most $m + \bar{n} + 1$ constraints, where $\bar{n} = \max_{q \in Q} n_q$. Hence, the complexity of the verification phase is polynomial in d, m, q and $\bar{n}$, and does not depends on the depth of the node.

Summarizing the above, we get the following complexity bound for Alg. 2.

Theorem 9: The complexity of Alg. 2 is

$$\mathcal{O}(m^{m^2 d^2 \log(1/\epsilon)} \mathrm{poly}(m, d, q, \bar{n})) \text{ FLOPs.}$$

Proof: Let $r = md$. With the use of the MVE to estimate to volume of the feasible set in the learning step, one must replace ϵ by ϵ^r and γ by $1 - 1/r$ in (8). Using the bound $-\log(1 - 1/r) \geq 1/r$, we get that $D \leq r^2 \log(\epsilon)$. The proof then follows from the observations above. ■

The complexity is exponential in d and m. This is to be expected since the problem of finding an m-piece polyhedral function satisfying the Lyapunov condition (2) for a finite set of pairs $\{(x_1, i_1), \ldots, (x_N, i_N)\} \subseteq \mathbb{R}_*^d \times [m]$ amounts to solve a Mixed-Integer Linear Program (MILP) with N integer variables. The complexity of such a program is exponential in N [24]. Finally, note that the approach proposed in this paper is better than the one consisting in solving the MILP for a finite set of pairs $\{(x_1, i_1), \ldots, (x_N, i_N)\} \subseteq \mathbb{R}_*^d \times [m]$ and adding recursively new pairs (x_{N+1}, i_{N+1}) to

this set in a counterexample-guided fashion until convergence to a valid Lyapunov function. Indeed, since the feasible set of an MILP is nonconvex, one cannot get guarantees on the decrease of the volume of the feasible set as in Lemma 7, so that the bound on the number $N_{\max}$ of pairs needed before the algorithm terminates is itself exponential in md, making the overall complexity (double-exponential) growing way too fast to be useful in practice.

IV. Numerical Example

Consider the piecewise linear system described by (1) with $Q = \{1, 2\}$, $H_1 = \mathbb{R} \times \mathbb{R}_{\geq 0}$ and $H_2 = \mathbb{R} \times \mathbb{R}_{\leq 0}$, and

$$A_1 = \begin{bmatrix} \frac{-1}{2} & 1 \\ -1 & \frac{-1}{2} \end{bmatrix}, \quad A_2 = \begin{bmatrix} \frac{-3}{4} & 1 \\ -1 & \frac{-3}{4} \end{bmatrix}.$$

Although a trivial quadratic Lyapunov function (e.g., $V_{\text{quad}} : x \mapsto \|x\|^2$) exists for this system, finding a polyhedral one is challenging because many pieces are required to capture the rotational dynamics of the system; see Figure 5-left for the vector field of the system, the 1.5-sublevel set of V_{quad} and a sample trajectory of the system.

Using the process described in Section III, we computed a 8-piece polyhedral Lyapunov function for the system. As an approximation of the MVE center, we used the Chebyshev center (center of the largest enclosed Euclidean ball), which can be computed using Linear Programming (more efficient and reliable than Semidefinite Programming). Although the theoretical guarantees on the termination of the process using the Chebyshev center are weaker than the ones with the MVE center (see Theorem 9), this provides a powerful heuristic in practice (see, e.g., [20, §4.4] and the references therein). The computation takes about 60 seconds.[2] The 1-sublevel set of the polyhedral Lyapunov function is represented in Figure 5-right (in yellow). The points "x" involved in the scenario $Y \in \mathbb{R}^d_* \times [m] \times [m]$ leading to this function are represented by blue dots. We have also represented a sample trajectory of the system (in purple). We observe that the trajectory does not leave the sublevel set, as predicted by the theory of Lyapunov.

V. Conclusions

In conclusion, we provide a NP-hardness result for finding fixed-complexity polyhedral Lyapunov function for linear hybrid systems in order to prove asymptotic stability. Furthermore, we describe a counterexample-driven approach based on switching between learning and verification followed by a tree of possible refutations. Our approach is modified to yield termination but at the cost of completeness. Our future work will focus on an efficient implementation of the procedure described in this paper and extensions to barrier and control Lyapunov function synthesis.

[2]On a laptop with processor Intel Core i7-7600u and 16 GB RAM running Windows. We use Gurobi™, under academic license, as linear optimization solver.

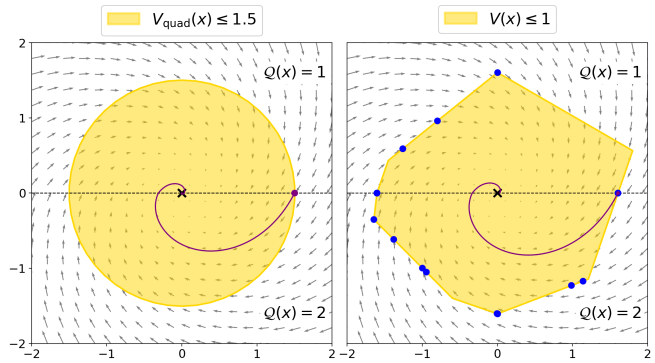

Fig. 5. Vector field (gray arrows) and sample trajectories (purple lines) of the system described in Section IV. *Left:* Quadratic Lyapunov function. *Right:* 8-piece polyhedral Lyapunov function. The blue dots are the points involved in the scenario leading to this polyhedral function.

Appendix

A. Proof of Theorem 2

We will show that m-Lyap-Fun-Exist with $m = 2$ is NP-hard by reducing NAE-SAT3 to it. The NAE-SAT3 (Not All Equal SATisfiability) is a version of propositional satisfiability problem that takes as input a Boolean formula φ in the conjunctive normal form (CNF) with propositions $p_1, \ldots, p_n$ and clauses $C_1, \ldots, C_L$. Each clause involves a disjunction of three literals: for each $l \in [L]$, $C_l = \bigvee_{h=1}^{3} s_{l,h} p_{\alpha_{l,h}}$ where for all $h \in [3]$, $s_{l,h} \in \{-1, 1\}$ and $\alpha_{l,h} \in [n]$, with the notation that for all $k \in [n]$, $-p_k$ denotes its negation $\neg p_k$.

Given such a formula, the NAE-SAT3 problem asks if there exists a truth assignment of *true* or *false* to each proposition p_k such that for each clause C_l at least one literal is true (the clause is satisfied), but at the same time, at least one literal is false. It is well known that the NAE-SAT3 problem is NP-complete [25].

We will now reduce any given NAE-SAT3 instance with n propositions $p_1, \ldots, p_n$ and L clauses $C_1, \ldots, C_L$ to yield a piecewise linear system (1) that has a polyhedral Lyapunov function of the form $\max(f^\top x, g^\top x)$ iff φ has a truth assignment that makes it NAE-SAT3.

Recall the condition (2) for $(c_i)_{i=1}^m \subseteq \mathbb{R}^d$ to yield a polyhedral Lyapunov function for System (1):

$$\begin{aligned} &(\forall\, q \in Q,\, x \in \mathcal{H}_q \setminus \{0\},\, i \in [m]) \\ &[((\forall\, j \in [m])\ c_i^\top x \geq c_j^\top x) \Rightarrow \\ &\qquad (c_i^\top x > 0 \wedge c_i^\top A_q x < 0)]. \end{aligned} \tag{9}$$

Let $d = n + 2$. Consider a vector of d variables: $x = (\bar{x}_1, \ldots, \bar{x}_n, \hat{x}, \tilde{x})$. Also, consider two vectors of d coefficients: $f = (\bar{f}_1, \ldots, \bar{f}_n, \hat{f}, \tilde{f})$ and $g = (\bar{g}_1, \ldots, \bar{g}_n, \hat{g}, \tilde{g})$. We are looking for a 2-piece polyhedral Lyapunov function, defined by $V(x) = \max\{f^\top x, g^\top x\}$, for the system defined below.

For each $k \in [n]$, let

$$\mathcal{H}_k = \{(\bar{x}, \hat{x}, \tilde{x}) : [(\forall\, k' \in [n] \setminus \{k\})\ \bar{x}_{k'} = 0] \wedge \hat{x} = \tilde{x} = 0\}$$

and consider the dynamics: $\dot{\bar{x}}_k = -\bar{x}_k$; for all $k' \in [n] \setminus \{k\}$, $\dot{\bar{x}}_{k'} = 0$; $\dot{\hat{x}} = 0$; $\dot{\tilde{x}} = |x_k|$ (mind the absolute value which

can be cast as a piecewise linear dynamics by splitting $\mathcal{H}_k$ in two pieces). The condition (9) then becomes:

$$\begin{aligned}&(\forall\, \lambda \neq 0)\\ &[(\bar{f}_k\lambda \geq \bar{g}_k\lambda \;\Rightarrow\; (\bar{f}_k\lambda > 0 \;\wedge\; -\bar{f}_k\lambda + \tilde{f}|\lambda| < 0))\\ &\quad \wedge (\bar{g}_k\lambda \geq \bar{f}_k\lambda \;\Rightarrow\; (\bar{g}_k\lambda > 0 \;\wedge\; -\bar{g}_k\lambda + \tilde{g}|\lambda| < 0))].\end{aligned}$$

This is equivalent to saying that (i) $\bar{f}_k\bar{g}_k < 0$ (i.e., both are nonzero and have opposite signs), (ii) $|\bar{f}_k| > \tilde{f}$, and (iii) $|\bar{g}_k| > \tilde{g}$.

If $\bar{f}_k < 0$ (i.e., $\bar{g}_k > 0$), we assign the value *true* to p_k; otherwise, we assign the value *false*.

Now, for each $l \in [m]$, let

$$\mathcal{H}_l = \{(\bar{x}, \hat{x}, \tilde{x}) : \bar{x} = 0,\ \tilde{x} = 0\}$$

and consider the dynamics: for all $h \in [3]$, $\dot{\bar{x}}_{\alpha_{l,h}} = s_{l,h}|\hat{x}|$; for all $k \in [n] \setminus \{\alpha_{l,h}\}_{h=1}^{3}$, $\dot{\bar{x}}_k = 0$; $\dot{\hat{x}} = 0$; $\dot{\tilde{x}} = -3|\hat{x}|$. The condition (9) then becomes:

$$\begin{aligned}&(\forall\, \lambda \neq 0)\\ &[(\hat{f}\lambda \geq \hat{g}\lambda \;\Rightarrow\; (\hat{f}\lambda > 0 \;\wedge\; \textstyle\sum_{h=1}^{3} s_{l,h}\bar{f}_{\alpha_{l,h}} - 3\tilde{f} < 0))\\ &\quad \wedge (\hat{g}\lambda \geq \hat{f}\lambda) \;\Rightarrow\; (\hat{g}\lambda > 0 \;\wedge\; \textstyle\sum_{h=1}^{3} s_{l,h}\bar{g}_{\alpha_{l,h}} - 3\tilde{g} < 0))].\end{aligned}$$

This is equivalent to saying that $\hat{f}\hat{g}$, $\sum_{h=1}^{3} s_{l,h}\bar{f}_{\alpha_{l,h}} - 3\tilde{f}$ and $\sum_{h=1}^{3} s_{l,h}\bar{g}_{\alpha_{l,h}} - 3\tilde{g}$ are negative.

Now, it holds that $\sum_{h=1}^{3} s_{l,h}\bar{f}_{\alpha_{l,h}} - 3\tilde{f} < 0$ only if there is $h \in [3]$ such that $s_{l,h}\bar{f}_{\alpha_{l,h}} < 0$. Similarly, $\sum_{h=1}^{3} s_{l,h}\bar{g}_{\alpha_{l,h}} - 3\tilde{g} < 0$ only if there is $h \in [3]$ such that $s_{l,h}\bar{g}_{\alpha_{l,h}} < 0$. Thus, NAE-SAT3 is satisfied with the assignment described above. On the other hand, if $p_1, \ldots, p_n$ satisfies NAE-SAT3, then the assignment: for all $k \in [n]$, $\bar{f}_k = -\bar{g}_h = 1$ if $\neg p_k$ and $\bar{f}_k = -\bar{g}_k = -1$ if p_k, $\hat{f} = -\hat{g} = 1$, and $\tilde{f} = \tilde{g} = 0.9$, provides a 2-piece polyhedral Lyapunov function for the above system. Thus, the above system admits a 2-piece polyhedral Lyapunov function iff NAE-SAT3 is satisfiable.

B. Further analysis of Example 1

We prove that System (1) with the parameters given in Example 1 does not admit a polynomial Lyapunov function.

For the sake of a contradiction, assume that V is a polynomial Lyapunov function for the system. Let $x_0 = [1, 0]^\top$, and let $p : \mathbb{R} \to \mathbb{R}$ be the univariate polynomial defined by $p(r) = V(rx_0)$. Since V is radially unbounded, it holds that p is of nonzero even degree. Fix $r \in \mathbb{R}_{>0}$, and let $x : \mathbb{R}_{\geq 0} \to \mathbb{R}^d$ be the system trajectory starting from $x(0) = rx_0$. It holds that for all $t \in [0, \pi]$, $x(t) = e^{A_2 t} r x_0 = e^{0.01t} r[\cos(t), -\sin(t)]^\top$. Hence, $V(x(0)) = p(r)$ and $V(x(\pi)) = p(-e^{0.01\pi} r)$. Since p is of nonzero even degree, it holds that for r large enough, $p(-e^{0.01\pi} r) > p(r)$, so that $V(x(\pi)) > V(x(0))$. This contradicts the property that V must decrease along the trajectories of the system [1, Theorem 4.1], concluding the proof.

References

[1] H. K. Khalil, *Nonlinear systems*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2002.

[2] D. Liberzon, *Switching in systems and control*. Boston, MA: Birkhäuser, 2003.

[3] Z. Sun and S. S. Ge, *Stability theory of switched dynamical systems*. London: Springer, 2011.

[4] F. Blanchini and S. Miani, *Set-theoretic methods in control*, 2nd ed. Cham: Birkhäuser, 2015.

[5] U. Topcu, A. Packard, and P. Seiler, "Local stability analysis using simulations and sum-of-squares programming," *Automatica*, vol. 44, no. 10, pp. 2669–2675, 2008.

[6] J. Kapinski, J. V. Deshmukh, S. Sankaranarayanan, and N. Aréchiga, "Simulation-guided Lyapunov analysis for hybrid dynamical systems," in *Proceedings of the 17th international conference on Hybrid systems: computation and control*. ACM, 2014, pp. 133–142.

[7] Y.-C. Chang, N. Roohi, and S. Gao, "Neural Lyapunov control," in *NIPS'19: Proceedings of the 33rd International Conference on Neural Information Processing Systems*. ACM, 2019, pp. 3245–3254.

[8] H. Ravanbakhsh and S. Sankaranarayanan, "Learning control Lyapunov functions from counterexamples and demonstrations," *Autonomous Robots*, vol. 43, no. 2, pp. 275–307, 2019.

[9] S. Miani and C. Savorgnan, "MAXIS-G: a software package for computing polyhedral invariant sets for constrained LPV systems," in *Proceedings of the 44th IEEE Conference on Decision and Control*. IEEE, 2005, pp. 7609–7614.

[10] N. Guglielmi, L. Laglia, and V. Protasov, "Polytope Lyapunov functions for stable and for stabilizable LSS," *Foundations of Computational Mathematics*, vol. 17, no. 2, pp. 567–623, 2017.

[11] A. Polański, "On absolute stability analysis by polyhedral Lyapunov functions," *Automatica*, vol. 36, no. 4, pp. 573–578, 2000.

[12] R. Ambrosino, M. Ariola, and F. Amato, "A convex condition for robust stability analysis via polyhedral Lyapunov functions," *SIAM Journal on Control and Optimization*, vol. 50, no. 1, pp. 490–506, 2012.

[13] D. Kousoulidis and F. Forni, "Polyhedral Lyapunov functions with fixed complexity," in *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 2021, pp. 3293–3298.

[14] J.-P. Aubin and A. Cellina, *Differential inclusions: set-valued maps and viability theory*. Berlin: Springer, 1984.

[15] S. Boyd and L. Vandenberghe, *Convex optimization*. Cambridge, UK: Cambridge University Press, 2004.

[16] K. P. Bennett and O. L. Mangasarian, "Bilinear separation of two sets in n-space," *Computational Optimization and Applications*, vol. 2, no. 3, pp. 207–227, 1993.

[17] J. Gronski, M.-A. Ben Sassi, S. Becker, and S. Sankaranarayanan, "Template polyhedra and bilinear optimization," *Formal Methods in System Design*, vol. 54, no. 1, pp. 27–63, 2019.

[18] Y. Nesterov and A. Nemirovskii, *Interior-point polynomial algorithms in convex programming*. Philadelphia, PA: SIAM, 1994.

[19] A. Ben-Tal and A. Nemirovski, *Lectures on modern convex optimization: analysis, algorithms, and engineering applications*. Philadelphia, PA: SIAM, 2001.

[20] S. Boyd and L. Vandenberghe, "Localization and cutting-plane methods," 2008, https://see.stanford.edu/materials/lsocoee364b/05-localization_methods_notes.pdf.

[21] D. J. Newman, "Location of the maximum on unimodal surfaces," *Journal of the ACM*, vol. 12, no. 3, pp. 395–398, 1965.

[22] S. P. Tarasov, L. G. Khachiyan, and I. I. Èrlikh, "The method of inscribed ellipsoids," *Soviet Mathematics Doklady*, vol. 37, no. 1, pp. 226–230, 1988.

[23] F. John, "Extremum problems with inequalities as subsidiary conditions," in *Studies and Essays Presented to R. Courant on his 60th Birthday, January 8, 1948*, R. Courant, Ed. New York, NY: Interscience Publishers, 1948.

[24] R. J. Vanderbei, *Linear programming: foundations and extensions*, 5th ed. Cham: Springer, 2020.

[25] C. H. Papadimitriou, *Computational Complexity*. Reading, MA: Addison-Wesley, 1994.